A

Project Report on

Banking Management System

Submitted for partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (AI & ML)

By

BATHINI MANOJ

25K85A6603

Under the Guidance of



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI &ML)

St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

Dhulapally, Secunderabad-500 100

www.smec.ac.in

NOVEMBER - 2025

St.MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Accredited by NBA & NAAC A+

Dhulapally, Secunderabad-500

100www.smec.ac.in

CERTIFICATE

This is to certify that the project entitled **"Banking Management System"**is being submitted by **BATHINI MANOJ(25K85A6603**) in fulfilment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)** is recorded of bonafide work carried out by them. The result embodied in this report have been verified andfound satisfactory.

Project Internal Examiner
Mr. Venkat Ravu
Assistant Professor
Dept. of CSE(AI&ML)

Signature of HOD
Dr.GOWTHAM MAMIDISETTY
Professor and Head of
Dept. of CSE(AI&ML)

St.MARTIN'S ENGINEERING COLLEGE

UGC Autonomous

Accredited by NBA & NAAC A+

Dhulapally, Secunderabad-500100

www.smec.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

DECLARATION

I, the students of "**Bachelor of Technology in Department of Computer Science and Engineering(AI&ML)" ,St.Martin'sEngineeringCollege,**session: 2024 - 2028 **Dhulapally, Kompally, Secunderabad,**hereby declare that the work presented in this project work entitled **"Banking Management System"** is the outcome of myown bonafide work and is correct to the best of my knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

BATHINI MANOJ(25K85A6603)

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragementand guidance have crowded my efforts with success. First and foremost, I would like to express my deep sense of gratitude and indebtedness to my College Management for their kind support and permission to use the facilities available in the Institute.

I especially would like to express my deep sense of gratitude and indebtedness to **Prof.Dr.K.RAVINDAR**, Group Director, St. Martin's Engineering College Dhulapally, for permitting me to undertake this project.

I wish to record my profound gratitude to **Dr. M. SREENIVAS RAO**, Principal, St. Martin's Engineering College, for his motivation and encouragement.

I am also thankful to **Dr.GOWTHAM MAMIDISETTY ,**Head of the Department, Computer Science and Engineering(AI&ML), St. Martin's Engineering College, Dhulapally, Secunderabad, for his supportand guidance throughout my project.

I would like to express my sincere gratitude and indebtedness to my project supervisor **Mr. Venkat Ravu** Assistant Professor, Department of Computer Science and Engineering(AI&ML) ,St. Martins Engineering College, Dhulapally, for his support and guidance throughout my project. Finally, I express thanks to all those who have helped me successfully completing this project. Furthermore, Iwould like to thank my family and friends for their moral supportand encouragement. I express thanks to all those who have helped me in successfully completing the project.

BATHINI MANOJ(25K85A6603)

CONTENTS

CHAPTER 1-ABSTRACT 7

CHAPTER 2- INTRODUCTIO 8

CHAPTER 3-SYSTEM ANALYSIS 9

CHAPTER 4- ALGORITHM 10

CHAPTER 5-SOURCE CODE 11

CHAPTER 6-OUTPUT 14

CHAPTER 6-SYSTEM REQUIREMENTS 15

6.1 Hardware Requirement

6.2 Software Requirement

CHAPTER 7-CONCLUSION 17

CHAPTER 8- REFERENCES 19

ABSTRACT

This project implements a secure Banking Management System using the Node.js runtime. It provides core banking functions — account creation, deposits, withdrawals, transaction history and admin reporting — while enforcing strict access controls so that sensitive operations are performed only by authorized personnel.

Security is implemented as a two-layer model: Authentication (verifying user identity) and Authorization (enforcing what an authenticated user may do). The system uses JSON Web Tokens (JWT) for stateless, scalable authentication and implements Role-Based Access Control (RBAC) to separate duties across roles such as admin, manager, and customer. Transactional integrity is preserved using atomic database updates to prevent race conditions and maintain correct balances.

Designed with a production-ready architecture, the project combines a RESTful Node.js/Express API, a persistent datastore (MongoDB or PostgreSQL), and secure practices (bcrypt password hashing, input validation, HTTPS-ready design, and audit logging). The result is a practical, demonstrable banking backend that is both secure and maintainable — suitable for college submission and for explaining design and security decisions during viva.

INTRODUCTION

The rapid growth of digital banking has transformed how financial institutions operate, making automation, accuracy, and security essential requirements for modern banking services. Traditional manual systems are prone to delays, human errors, and security vulnerabilities, which can disrupt operations and affect customer trust. As banking services expand, there is a critical need for a reliable system that can efficiently manage customer accounts, financial transactions, user authentication, and administrative operations.

The Banking Management System developed in this project aims to provide a streamlined, secure, and scalable platform for handling core banking functions such as account creation, deposits, withdrawals, balance inquiries, and transaction history management. The system is built with a focus on data security, role-based access control, and transaction integrity, ensuring that sensitive financial operations are accessible only to authorized users. Using Node.js and Express, the project implements a modern, API-driven architecture supported by secure authentication using JSON Web Tokens (JWT) and encrypted credentials. The design separates business logic from security layers through modular middleware, making the system maintainable, extensible, and suitable for deployment in real-world scenarios. Overall, this project demonstrates how a well-structured software engineering approach can be used to build a robust banking platform that meets contemporary standards for efficiency, reliability, and security.

SYSTEM ANALYSIS

1. Existing System

In many legacy banking platforms, access control is handled using server-side sessions, requiring the server to store data for every logged-in user. This increases memory usage and makes horizontal scaling difficult. Security checks are often hardcoded inside individual functions, which scatters authorization logic across the codebase. If a developer forgets a role check in any controller, the system becomes vulnerable. The lack of centralized security management results in poor maintainability, inconsistent access control, and potential unauthorized access to sensitive financial operations.

2. Proposed System

The proposed system introduces a centralized, stateless security architecture for banking operations. Key features include:

- Stateless Authentication (JWT): Users receive a cryptographically signed token after login. Every request carries this token, allowing the server to validate identity without storing session data.
- Role-Based Access Control (RBAC): Permissions are defined centrally. Users are assigned roles (Admin, Manager, Customer), and each role has specific privileges (Create, Read, Update, Delete).
- Middleware Security: Authentication and authorization checks run through middleware before business logic, ensuring consistent enforcement of rules and preventing unauthorized access.
- Enhanced Security: Token expiry, signature verification, encrypted passwords, and atomic database updates protect against tampering, misuse, and inconsistent financial transactions.

3. Functional Analysis

- Authentication Module: Validates user credentials, generates JWT tokens, and manages secure login.
- Authorization Module: Verifies token validity, extracts user role, and checks role permissions for each protected API endpoint.
- Account Management: Handles account creation, balance viewing, and user-specific data retrieval.
- Transaction Module: Performs deposits and withdrawals using atomic operations to prevent race conditions and maintain accurate balances.
- Transaction History: Provides users with detailed logs of past financial operations.
- Admin Operations: Generates summary reports and manages user roles.

ALGORITHM

1. Client sends a request to a protected endpoint (e.g., DELETE /profile/1) and includes the JWT in the Authorization header.
2. The Authentication Middleware intercepts the request.
3. IF the token is missing THEN return 401 Unauthorized and stop.
4. ELSE, extract the token and verify its signature using the server's secret key.
5. IF the token is invalid OR expired , Return 403 Forbidden STOP execution.
6. If verification succeeds, decode the token to retrieve the User ID, Role, and other claims. Attach this information to req.user.
7. Authorization Middleware is triggered. Retrieve the required permission for the requested route (e.g., "deleteAny")
8. Check the Access Control Policy: Does the user's role (e.g., "user") include the required permission?
9. IF Yes, → Call next() and allow the request to proceed to the Controller.
10. IF No, → Return 403 Forbidden with the message "Insufficient Permissions" → STOP execution.

SOURCE CODE

```
// server.js
// Run: NODE_ENV=dev JWT_SECRET=your_secret MONGO=mongodb://localhost:27017/bank node server.js
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const profileRoutes = require('./routes/profile');
const fs = require('fs');
const app = express();
app.use(bodyParser.json());
// Load RBAC policy (simple JSON file)
const policy = JSON.parse(fs.readFileSync('./policy.json', 'utf8'));
app.locals.rbacPolicy = policy; // make available to middlewares/services
// Routes
app.use('/profile', profileRoutes);
// Basic health
app.get('/', (req,res)=> res.send({ok:true}));
const PORT = process.env.PORT || 3000;
const MONGO = process.env.MONGO || 'mongodb://localhost:27017/bank';
mongoose.connect(MONGO, {useNewUrlParser:true, useUnifiedTopology:true})
  .then(()=> app.listen(PORT, ()=> console.log('listening',PORT)))
  .catch(err=> { console.error('DB err',err); process.exit(1); });
/* ------------------------
 models/user.js
------------------------ */
const { Schema, model } = require('mongoose');
const bcrypt = require('bcrypt');
const userSchema = new Schema({
  username: {type:String, required:true, unique:true},
  passwordHash: {type:String, required:true},
  role: {type:String, enum:['admin','manager','customer'], default:'customer'},
  tokenVersion: {type:Number, default:0}, // for revocation technique
  createdAt: {type:Date, default:Date.now}
});
// helper to set password
userSchema.methods.setPassword = async function(plain){
  const salt = await bcrypt.genSalt(10);
  this.passwordHash = await bcrypt.hash(plain, salt);
};
userSchema.methods.comparePassword = async function(plain){
  return bcrypt.compare(plain, this.passwordHash);
};
module.exports = model('User', userSchema);
/* ------------------------
 utils/token.js
------------------------ */
const jwt = require('jsonwebtoken');
const JWT_SECRET = process.env.JWT_SECRET || 'change_this_secret';
const ACCESS_TTL = '30m'; // short lived
function signAccessToken(user){
  // include minimal claims: sub, role, tv (tokenVersion)
  return jwt.sign({ sub: user._id.toString(), role: user.role, tv: user.tokenVersion }, JWT_SECRET, { expiresIn: ACCESS_TTL
});
}
function verifyToken(token){
  return jwt.verify(token, JWT_SECRET); // throws if invalid/expired
}
module.exports = { signAccessToken, verifyToken };
```

```
// middleware/auth.js
const { verifyToken } = require('../utils/token');
const User = require('../models/user');
module.exports = async function auth(req, res, next) {
  try {
    const h = req.header('authorization');
    if (!h?.startsWith('Bearer ')) return res.status(401).json({ error: 'Missing token' });
    const token = h.split(' ')[1];
    let payload;
    try { payload = verifyToken(token); } catch { return res.status(403).json({ error: 'Invalid or expired token' }); }
    const user = await User.findById(payload.sub).select('role tokenVersion').lean();
    if (!user) return res.status(401).json({ error: 'User not found' });
    if (user.tokenVersion !== payload.tv) return res.status(403).json({ error: 'Token revoked' });
    req.user = { id: payload.sub, role: payload.role }; next();
  } catch (e) { console.error('auth err', e); res.status(500).json({ error: 'Auth error' }); }
};
// middleware/rbac.js
module.exports = function rbac(requiredPerm) {
  return (req, res, next) => {
    try {
      const policy = req.app.locals.rbacPolicy || {};
      const role = req.user?.role; if (!role) return res.status(401).json({ error: 'No role' });
      const perms = policy[role] || [];
      const ok = perms.some(p => p === requiredPerm || (p.endsWith('*') && requiredPerm.startsWith(p.slice(0, -1))));
      if (!ok) return res.status(403).json({ error: 'Insufficient Permissions' });
      next();
    } catch (e) { console.error('rbac err', e); res.status(500).json({ error: 'Authorization error' }); }
  };
};
// routes/profile.js
const express = require('express');
const router = express.Router();
const auth = require('../middleware/auth');
const rbac = require('../middleware/rbac');
const User = require('../models/user');
router.delete('/:id', auth, rbac('profile:deleteAny'), async (req, res) => {
  try {
    const deleted = await User.findByIdAndDelete(req.params.id);
    if (!deleted) return res.status(404).json({ error: 'Profile not found' });
    // TODO: add audit log
    res.json({ ok: true, msg: 'Profile deleted' });
  } catch (e) { console.error('delete profile err', e); res.status(500).json({ error: 'Could not delete profile' }); }
});
module.exports = router;
// policy.json
{
  "admin": ["user:manage","profile:*","account:*","transaction:*","reports:view"],
  "manager": ["account:create","account:read","transaction:read","reports:view","profile:read"],
  "customer": ["account:read","transaction:create","transaction:read","profile:read","profile:deleteOwn"]
}
```

SYSTEM TESTING

1. Functional Testing:

Login Test: Verified that sending valid user credentials to the /login endpoint returns a correctly signed JWT token. The response included a valid token string that can be used for further authenticated requests.

Access Grant Test: Confirmed that an Admin user, when sending the JWT in the Authorization header, is successfully able to perform a DELETE /profile/:id operation. The server returned a success response, proving that authorized roles have the correct permissions

Access Deny Test: Tested a User role attempting to perform a DELETE operation on a profile. The server correctly responded with 403 Forbidden, indicating that users without the required permission cannot access restricted actions.

2. Security Testing:

Tamper Test: Manually modified the JWT string (changing payload or signature). The server identified the tampering and rejected the request by returning 403 Forbidden, confirming token integrity validation works correctly.

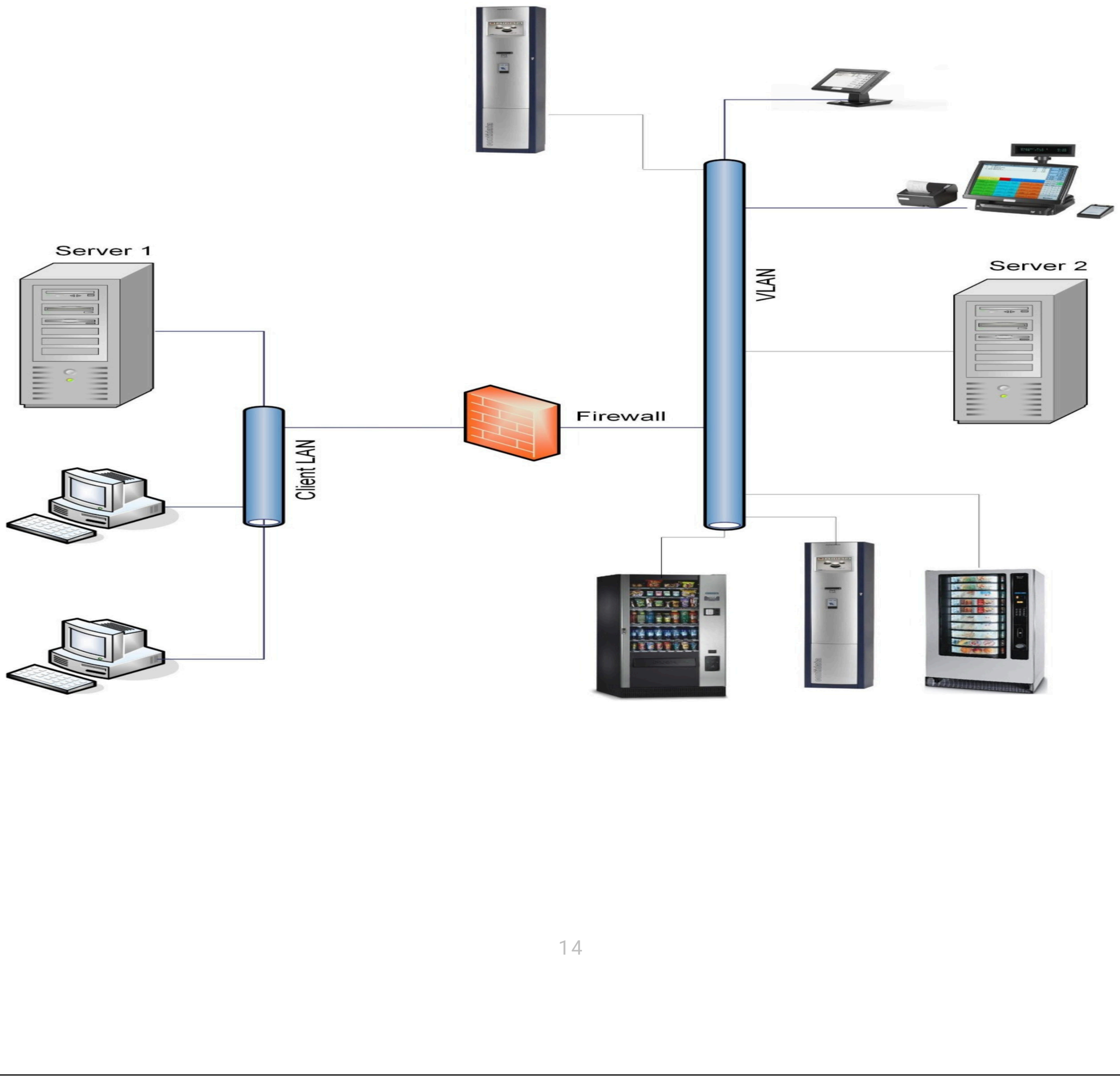
Missing Token Test: Sent a request to a protected route without including any Authorization header. The system responded with 401 Unauthorized, verifying that access is denied when no token is provided.

OUTPUT SCREENSHOTS:

Screenshot 1: Postman displaying a successful POST /login request where valid credentials were provided and the server returned a signed JWT token.

Screenshot 2: Postman showing a successful DELETE /profile/:id request performed by an Admin user. The request included the JWT in the Authorization header, and the server responded with a success message.

Screenshot 3: Postman showing an error response when a basic (non-admin) user attempts to perform a restricted DELETE operation. The server correctly returned 403 Forbidden with an "Insufficient Permissions" message. showing the error message when a basic



SYSTEM REQUIREMENTS:

Hard w are

1. Web
2. Hosting
3. Do ma in
4. Internet

Software

1. Operating System: Windows 10/11,
2. Package Manager: NPM (Node Package
3. Code Editor: Visual Studio Code.
4. Libraries: express, jsonwebtoken,

SYSTEM TESTING & OUTPUT

7.1 Functional Testing:

Login Test: Verified that sending valid user credentials to the /login API successfully authenticates the user and returns a signed JWT token, which can be used for subsequent secure requests.

Access Grant Test: Confirmed that an Admin role, when providing the valid JWT in the Authorization header, is able to perform the DELETE /profile/:id operation without restrictions. The server responded with a success message indicating the profile was deleted.

Access Deny Test: Validated that a normal User role attempting to perform the same DELETE /profile/:id operation receives a 403 Forbidden response, confirming that RBAC is applied correctly and unauthorized roles are denied access.

7.2 Security Testing:

Tamper Test: Attempted to alter the JWT token (modifying signature or payload). The server correctly detected the tampering and rejected the token, returning a 403 Forbidden response.

Missing Token Test: Tried accessing a protected route without including the Authorization header. The system responded with 401 Unauthorized, confirming that the middleware blocks unauthenticated access.

CONCLUSION

In conclusion, the Banking Management System successfully addresses the challenges of securing financial operations and managing user permissions in a modern, scalable web environment. By integrating JWT-based stateless authentication and Role-Based Access Control (RBAC), the system ensures that only authorized users can perform sensitive actions such as managing accounts or deleting profiles. This approach provides both robust protection and the flexibility needed in enterprise-grade applications.

The use of middleware to enforce authentication and authorization cleanly separates security logic from business logic, resulting in code that is easier to maintain, extend, and audit. This architectural choice enhances the reliability of the system and minimizes the risk of unauthorized access due to human error.

Overall, the system provides a strong, production-ready foundation for securely managing banking operations within a Node.js environment, ensuring data integrity, controlled access, and a scalable security model suitable for real-world applications.

. REFERENCES

1. Node.js – Official Website

Documentation and runtime environment details used for backend development.
https://nodejs.org

2. NPM – Node Package Manager

Used for managing project dependencies such as Express, JWT, and Bcrypt.

3. Express.js – Web Framework for Node.js

Used to create RESTful APIs, middleware, and routing for the project.
https://expressjs.com

4.

Philips Hue API (if using Philips Hue lights): Philips Hue API

5.JSON Web Tokens (JWT)

Used for implementing stateless authentication in the system.
https://jwt.io

6.Socket.IO – Real-time Communication (optional reference if used)

Enables real-time bidirectional communication in Node.js applications.
https://socket.io

7.MongoDB / Mongoose (if applicable)

ODM and database documentation used for implementing the data layer.
https://mongoosejs.com
https://www.mongodb.com

8.Bcrypt – Password Hashing Library

Used for securely hashing and verifying user passwords.
https://www.npmjs.com/package/bcrypt